

HSBI — Bielefeld University of Applied Sciences and Arts

Quantitative Research Methods

Summer Semester 2026 · Examiner: Prof. Dr. Christoph Weisser

Final Mock Examination — Set A (Lectures 1–12)

Comprehensive coverage of ISLP Chapters 2–8, 10 and 13,
with emphasis on Chapters 7 (splines/GAMs), 8 (trees), 10 (deep learning) and 13 (multiple
testing)

— Version with detailed solutions —

Duration: 120 minutes

Total credit: 120 points

Allowed aids: non-programmable calculator; one handwritten A4 sheet of notes (both sides)

Name: _____

Matriculation no.: _____

Problem	P1	P2	P3	P4	P5	P6	P7	Total
Maximum points	16	8	20	16	20	20	20	120
Achieved points								

Instructions

- Justify all answers. Numerical results without a derivation or a brief reasoning receive no credit.
- Round final numerical results to 3 decimal places unless stated otherwise. Small deviations caused by carrying rounded intermediate results are accepted.
- All numerical constants you need (powers of e , logarithms, $(0.95)^{25}$, t -quantiles) are provided in the problems.
- The number of points per problem roughly equals the recommended working time in minutes.

Problem 1: Moving beyond linearity — splines

(16 points)

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

- (a) [4 P] A *cubic spline* is a piecewise cubic polynomial that is continuous and has continuous first and second derivatives at each knot. How many degrees of freedom (free parameters, counting the intercept) does a cubic spline with K interior knots use? Justify your answer by counting the parameters of the polynomial pieces and the constraints at the knots, and evaluate the result for $K = 5$.
- (b) [4 P] How many degrees of freedom does a *natural* cubic spline with the same K knots use (again evaluate for $K = 5$)? Which additional constraint does a natural spline impose, in which region of the predictor space does it act, and why is this constraint useful in practice?
- (c) [4 P] A *smoothing spline* is the function g that minimises

$$\sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int g''(t)^2 dt.$$

Explain the role of the two terms, and describe the behaviour of the solution $\hat{g}$ in the two limiting cases $\lambda \rightarrow 0$ and $\lambda \rightarrow \infty$.

- (d) [4 P] Write down the *truncated-power basis* representation of a cubic spline with a single knot at ξ , define the truncated-power basis function explicitly, and explain in one sentence why adding this basis function does not destroy the smoothness of the fit at ξ .

Solution

(a) Degrees of freedom of a cubic spline. K interior knots split the range of X into $K + 1$ intervals, each carrying a cubic polynomial with 4 coefficients: $4(K + 1) = 4K + 4$ parameters. At each knot, continuity of g , g' and g'' imposes 3 constraints, i.e. $3K$ constraints in total. Hence

$$\text{df} = 4(K + 1) - 3K = K + 4.$$

For $K = 5$: $\text{df} = 5 + 4 = 9$ free parameters (equivalently: intercept, x , x^2 , x^3 plus one truncated-power basis function per knot $= 4 + K$).

(b) Natural cubic spline. A natural spline additionally requires the function to be *linear beyond the boundary knots*, i.e. in the two outer regions $X < \xi_1$ and $X > \xi_K$. Forcing the quadratic and cubic terms to vanish there removes 2 parameters per boundary region, i.e. 4 in total:

$$\text{df} = (K + 4) - 4 = K, \quad K = 5 : \text{df} = 5.$$

Usefulness: polynomials and unconstrained splines are notoriously erratic near the boundaries, where data are sparse; the linearity constraint stabilises the fit there (*lower variance* of predictions at and beyond the boundary), at the price of a small amount of additional bias.

(c) Smoothing spline. The first term (RSS) rewards fidelity to the data; the second term penalises roughness, since g'' measures the curvature (wiggleness) of g , and $\lambda \geq 0$ is the tuning parameter that trades the two off. $\lambda \rightarrow 0$: the penalty disappears; $\hat{g}$ can be made arbitrarily wiggly and *interpolates* the training observations exactly (zero training RSS, severe overfitting; effective $\text{df} \rightarrow n$). $\lambda \rightarrow \infty$: any curvature is infinitely expensive, so $g'' \equiv 0$ is enforced; $\hat{g}$ becomes a straight line and equals the *least-squares regression line* (effective $\text{df} \rightarrow 2$). Between the extremes, λ (chosen e.g. by LOOCV) controls the effective degrees of freedom, which decrease smoothly from n to 2.

(d) Truncated-power basis with one knot ξ .

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3, \quad (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi \\ 0, & x \leq \xi \end{cases}$$

The function $(x - \xi)_+^3$ and its first and second derivatives are all 0 at $x = \xi$, so adding it changes only the third derivative at the knot; the fit remains continuous with continuous first and second derivatives — exactly the defining smoothness of a cubic spline.

Problem 2: Generalised additive models

(8 points)

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

- (a) [3 P] Write down the general form of a *generalised additive model* (GAM) for a quantitative response Y and predictors $X_1, \dots, X_p$, and explain in one sentence how it extends multiple linear regression.
- (b) [3 P] State one important *advantage* of GAMs that concerns the interpretation of the fitted

functions f_j , and the main structural *limitation* of GAMs — including how this limitation can be repaired.

- (c) [2 P] Describe in one or two sentences how the *backfitting* algorithm fits a GAM.

Solution

(a) **Model form.**

$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p) + \varepsilon.$$

Multiple linear regression is the special case $f_j(X_j) = \beta_j X_j$; a GAM replaces each linear component by a smooth non-linear function f_j (e.g. a natural or smoothing spline, or local regression), while keeping the *additive* structure so that the effects still simply add up.

(b) **Advantage and limitation.** *Advantage (interpretability):* because the model is additive, the effect of each X_j on Y is summarised by its own curve f_j , which can be plotted and read one predictor at a time while the others are held fixed — non-linear effects stay as easy to inspect as a coefficient in a table. *Limitation:* additivity rules out *interactions*: the effect of X_j is not allowed to depend on the value of X_k . Repair: add interaction terms by hand, e.g. a product $X_j \times X_k$ or a low-dimensional joint smooth $f_{jk}(X_j, X_k)$ as an extra component.

(c) **Backfitting.** Cycle through the predictors one at a time and repeatedly re-estimate each f_j (with a smoother, e.g. a spline) from the *partial residuals* $r_i = y_i - \beta_0 - \sum_{k \neq j} f_k(x_{ik})$, holding all other fitted functions fixed; iterate this sweep until the f_j no longer change.

Problem 3: Regression and classification trees by hand

(20 points)

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

A café chain records for $n = 6$ branches the number of in-store promotional events x run in a month and the corresponding monthly revenue y (in 1,000 EUR):

Branch i	1	2	3	4	5	6
Events x_i	1	2	3	4	5	6
Revenue y_i	4	6	8	14	16	18

A regression tree with a *single* split at cutpoint s partitions the data into $R_1 = \{i : x_i < s\}$ and $R_2 = \{i : x_i \geq s\}$ and predicts the region mean $\hat{y}_{R_m}$ in each region.

- (a) [8 P] For each of the two candidate cutpoints $s = 2.5$ and $s = 3.5$, determine the two regions, the region means and the resulting total residual sum of squares $\text{RSS}(s) = \sum_{m=1}^2 \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2$. Which cutpoint does recursive binary splitting choose, and why?
- (b) [2 P] Using the split at $s = 3.5$ from part (a) (so $R_2 = \{x \geq 3.5\}$), predict the monthly revenue for a new branch that runs $x = 5$ promotional events.
- (c) [6 P] A *classification* tree for a churn task has a terminal node containing 10 training customers: 7 of class “Stay” and 3 of class “Leave”. Compute the class proportions, the Gini index $G = \sum_k \hat{p}_k(1 - \hat{p}_k)$ and the entropy $D = -\sum_k \hat{p}_k \ln \hat{p}_k$ of this node (use $\ln 0.7 = -0.3567$ and $\ln 0.3 = -1.2040$). Which class does the node predict?
- (d) [4 P] In *cost-complexity pruning* one minimises, for a subtree $T \subseteq T_0$ of the large tree T_0 ,

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|.$$

Explain the role of the tuning parameter α : what happens for $\alpha = 0$, what happens as α increases, and how is α chosen in practice?

Solution

(a) Evaluating the two cutpoints. Root node for reference: $\bar{y} = 66/6 = 11$.

Cutpoint $s = 2.5$: $R_1 = \{1, 2\}$ with y -values $\{4, 6\}$, mean $\hat{y}_{R_1} = 5$; $R_2 = \{3, 4, 5, 6\}$ with y -values $\{8, 14, 16, 18\}$, mean $\hat{y}_{R_2} = 56/4 = 14$.

$$\text{RSS}_{R_1} = (4 - 5)^2 + (6 - 5)^2 = 2,$$

$$\text{RSS}_{R_2} = (8 - 14)^2 + (14 - 14)^2 + (16 - 14)^2 + (18 - 14)^2 = 36 + 0 + 4 + 16 = 56,$$

so $\text{RSS}(2.5) = 2 + 56 = \mathbf{58}$.

Cutpoint $s = 3.5$: $R_1 = \{1, 2, 3\}$ with y -values $\{4, 6, 8\}$, mean $\hat{y}_{R_1} = 18/3 = 6$; $R_2 = \{4, 5, 6\}$ with y -values $\{14, 16, 18\}$, mean $\hat{y}_{R_2} = 48/3 = 16$.

$$\text{RSS}_{R_1} = (4 - 6)^2 + (6 - 6)^2 + (8 - 6)^2 = 4 + 0 + 4 = 8,$$

$$\text{RSS}_{R_2} = (14 - 16)^2 + 0 + (18 - 16)^2 = 4 + 0 + 4 = 8,$$

so $\text{RSS}(3.5) = 8 + 8 = \mathbf{16}$.

Since $16 < 58$, recursive binary splitting is *greedy* and picks the cutpoint with the smallest RSS: $s = \mathbf{3.5}$. (It cleanly separates the three low-revenue branches, mean 6, from the three high-revenue branches, mean 16.)

(b) Prediction for $x = 5$. $x = 5 \geq 3.5$, so the new branch falls into R_2 and the tree predicts $\hat{y} = \hat{y}_{R_2} = \mathbf{16}$ (i.e. 16,000 EUR of monthly revenue).

(c) Node impurity. $\hat{p}_{\text{Stay}} = 7/10 = 0.7$, $\hat{p}_{\text{Leave}} = 3/10 = 0.3$.

$$G = 0.7 \cdot 0.3 + 0.3 \cdot 0.7 = 0.21 + 0.21 = \mathbf{0.42}.$$

$$D = -(0.7 \ln 0.7 + 0.3 \ln 0.3) = -(0.7 \cdot (-0.3567) + 0.3 \cdot (-1.2040)) = 0.2497 + 0.3612 = \mathbf{0.611}.$$

The node predicts the majority class, **Stay** (with estimated probability 0.7). Both G and D are 0 for a pure node and largest at a 50:50 split; here the node is moderately impure.

(d) Role of α . α is the price per terminal node: it trades off the fit of the subtree (first term) against its complexity $|T|$. For $\alpha = 0$ the criterion equals the training RSS and the full tree T_0 is optimal. As α increases, additional leaves must “pay for themselves”; branches whose RSS reduction is smaller than α are pruned, so the optimal subtree becomes smaller — one obtains a *nested sequence* of subtrees as α grows (weakest-link pruning). In practice α is selected by *cross-validation* (choose the α with the smallest CV error, then refit on all data), which balances bias against variance instead of optimising training fit.

Problem 4: Ensembles — bagging, random forests, boosting

(16 points)

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

- (a) **[6 P]** Let $\hat{f}_1(x), \dots, \hat{f}_B(x)$ be B identically distributed (but not independent) tree predictions at a fixed point x , each with variance σ^2 and with pairwise correlation ρ . The variance of the bagged average is

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)\right) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2.$$

- (i) Explain briefly why this formula shows that bagging reduces variance, and which term limits the reduction. (ii) Evaluate the variance for $B = 200$, $\rho = 0.5$ and $\sigma^2 = 9$, and compare it with the variance of a single tree. (iii) What value does the variance approach as $B \rightarrow \infty$?
- (b) [4 P] How do *random forests* modify bagging at each split, and what is the recommended subset size m for *regression* as a function of p (evaluate for $p = 15$)? Explain, with reference to the variance formula $\text{Var} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ from part (a), why this modification improves on bagging.
- (c) [4 P] The following `scikit-learn` code fits a boosted regression-tree model:
- ```
from sklearn.ensemble import GradientBoostingRegressor
gbr = GradientBoostingRegressor(
 n_estimators=500, learning_rate=0.01, max_depth=2)
gbr.fit(X_train, y_train)
```
- The three keyword arguments `n_estimators`, `learning_rate` and `max_depth` are exactly the three tuning parameters of boosting. State in one sentence each what each of them controls, and explain the “slow learning” idea behind boosting — in particular why one chooses a small `learning_rate` such as 0.01.
- (d) [2 P] What does the *out-of-bag* (OOB) error of a bagged model estimate, and why is it available essentially for free?

### Solution

**(a) Variance of the bagged average.** (i) The second term  $\frac{1-\rho}{B}\sigma^2$  is driven to 0 by averaging over many trees ( $B$  large), so averaging removes the “independent part” of the trees’ variability — this is the variance reduction of bagging. The first term  $\rho\sigma^2$  does *not* depend on  $B$ : because the bootstrap trees are grown on overlapping samples of the same data (and share the same strong predictors), they are positively correlated, and this correlated part of the variance cannot be averaged away. It limits the reduction. (ii) For  $B = 200$ ,  $\rho = 0.5$ ,  $\sigma^2 = 9$ :

$$\text{Var} = 0.5 \cdot 9 + \frac{1 - 0.5}{200} \cdot 9 = 4.5 + \frac{0.5 \cdot 9}{200} = 4.5 + 0.0225 = \mathbf{4.5225},$$

compared with  $\sigma^2 = 9$  for a single tree — a reduction of about 50% ( $\frac{9-4.5225}{9} = 49.75\%$ ). (iii) As  $B \rightarrow \infty$  the variance approaches the floor  $\rho\sigma^2 = 0.5 \cdot 9 = \mathbf{4.5}$ . (Increasing  $B$  further does not overfit; it only stabilises the average.)

**(b) Random forests.** At each split, only a *random subset* of  $m$  of the  $p$  predictors is considered as splitting candidates; a fresh subset is drawn at every split. Recommended for regression:  $m \approx p/3$ , so for  $p = 15$ :  $m = 15/3 = \mathbf{5}$  (for classification one instead uses  $m \approx \sqrt{p}$ ). Why it helps: with bagging, a dominant predictor is used in the top split of almost every tree, so the trees look alike and  $\rho$  is high. Restricting the candidate set excludes that predictor from roughly  $(p - m)/p$  of the split searches and *decorrelates* the trees, i.e. lowers  $\rho$  — and by (a) this lowers the irreducible floor  $\rho\sigma^2$  of the ensemble variance, which mere averaging cannot touch.

**(c) Boosting.** The three keyword arguments *are* the three tuning parameters:

- `n_estimators` =  $B$ , the *number of trees* added sequentially; unlike bagging, boosting *can* overfit if  $B$  is too large, so  $B$  is chosen by cross-validation.
- `learning_rate` =  $\lambda$ , the *shrinkage* (here 0.01): it scales the contribution of each new tree and thus the speed of learning; smaller  $\lambda$  usually needs larger  $B$ .
- `max_depth` =  $d$ , the *number of splits / interaction depth* per tree (here 2): the complexity of each tree and the interaction order of the model; often even  $d = 1$  (stumps, additive model) works well.

*Slow learning:* each tree is fitted to the current *residuals*, and only a small fraction  $\lambda$  of it is added to the model; the model thus improves slowly and exactly in the regions where it still

performs poorly, instead of fitting the data hard in one step — which tends to generalise better. Choosing a small `learning_rate` such as 0.01 is precisely how this slow learning is implemented (at the cost of needing more trees, i.e. a larger `n_estimators`).

(d) **OOB error.** Each bootstrap sample leaves out on average about 1/3 of the observations (“out-of-bag”). Predicting each observation only with the trees that did *not* use it, and aggregating these predictions, yields the OOB error — a valid estimate of the *test error* of the bagged model. It is free because it reuses the bootstrap side product; no separate validation set or cross-validation loop is required.

## Problem 5: Neural networks by hand

(20 points)

Lecture slides: Chapter 10 — Deep Learning (Lecture 11).

Consider a tiny feed-forward network for a quantitative response with input  $x = (x_1, x_2) = (2, 3)$ , one hidden layer with two ReLU units ( $g(z) = \max\{0, z\}$ ) and a linear output unit:

$$A_1 = g(1 + 2x_1 - 1x_2), \quad A_2 = g(-2 - 1x_1 + 1x_2), \quad f(x) = -1 + 3A_1 + 2A_2.$$

- [8 P]** Perform the forward pass step by step: compute both pre-activations, both activations  $A_1, A_2$  and the network output  $f(x)$ . Comment briefly on what the ReLU did to the second unit and why such non-linear activations are essential.
- [6 P]** A classification network with  $K = 3$  classes produces the logits (output-layer pre-activations)  $z = (z_1, z_2, z_3) = (2.0, 1.0, -1.0)$  for one observation whose true class is class 1. Compute all three softmax probabilities and the cross-entropy loss  $-\ln \hat{p}_{\text{true}}$ . (Use  $e^2 = 7.389$ ,  $e^1 = 2.718$ ,  $e^{-1} = 0.368$  and  $\ln 10.475 = 2.349$ .)
- [4 P]** Count the parameters of a fully connected network with  $p = 5$  inputs, one hidden layer with  $k = 4$  units and an output layer with  $K = 3$  units, where every hidden and output unit has a bias. Show the count separately for the two weight layers.
- [2 P]** A convolutional layer receives a  $28 \times 28$  image and uses filters of size  $F = 3$  with padding  $P = 1$  and stride  $S = 1$ . Compute the spatial size of the output feature map from  $(W - F + 2P)/S + 1$ .

### Solution

(a) **Forward pass.** Pre-activations:

$$z_1 = 1 + 2 \cdot 2 - 1 \cdot 3 = 1 + 4 - 3 = 2, \quad z_2 = -2 - 1 \cdot 2 + 1 \cdot 3 = -2 - 2 + 3 = -1.$$

ReLU activations:

$$A_1 = g(2) = \max\{0, 2\} = 2, \quad A_2 = g(-1) = \max\{0, -1\} = 0.$$

Output:

$$f(x) = -1 + 3 \cdot 2 + 2 \cdot 0 = -1 + 6 + 0 = 5.$$

The ReLU *switched the second unit off*: its negative pre-activation is clipped to 0, so unit 2 contributes nothing for this input (but would for other inputs). This thresholding is the source of non-linearity: without a non-linear  $g$ , the network would collapse into a single linear function of  $x$ , no matter how many layers it has.

(b) **Softmax and cross-entropy.** Exponentials:  $e^{2.0} = 7.389$ ,  $e^{1.0} = 2.718$ ,  $e^{-1.0} = 0.368$ ; their sum is  $7.389 + 2.718 + 0.368 = 10.475$ . Softmax probabilities:

$$\hat{p}_1 = \frac{7.389}{10.475} = 0.705, \quad \hat{p}_2 = \frac{2.718}{10.475} = 0.259, \quad \hat{p}_3 = \frac{0.368}{10.475} = 0.035,$$

which sum to 1.000. Cross-entropy loss for true class 1:

$$-\ln \hat{p}_1 = -\ln \frac{e^2}{10.475} = -(2 - \ln 10.475) = \ln 10.475 - 2 = 2.349 - 2 = \mathbf{0.349}.$$

(Equivalently  $-\ln 0.705 \approx 0.349$ ; a perfect prediction  $\hat{p}_1 = 1$  would give loss 0.)

(c) **Parameter count for  $5 \rightarrow 4 \rightarrow 3$ .** Input  $\rightarrow$  hidden:  $5 \cdot 4 = 20$  weights + 4 biases = 24. Hidden  $\rightarrow$  output:  $4 \cdot 3 = 12$  weights + 3 biases = 15. Total:  $24 + 15 = \mathbf{39}$  parameters.

(d) **Convolution output size.**

$$\frac{W - F + 2P}{S} + 1 = \frac{28 - 3 + 2 \cdot 1}{1} + 1 = \frac{27}{1} + 1 = 28,$$

so the output feature map is  $\mathbf{28 \times 28}$  — padding  $P = 1$  with  $F = 3$ ,  $S = 1$  is exactly the “same” convolution that preserves the spatial size.

## Problem 6: Multiple testing

(20 points)

Lecture slides: Chapter 13 — Multiple Testing (Lecture 12).

- (a) [4 P] A lab performs  $m = 25$  independent hypothesis tests, each at level  $\alpha = 0.05$ , and suppose all 25 null hypotheses are true. Define the family-wise error rate (FWER) and compute it for this situation (use  $(0.95)^{25} = 0.2774$ ). Interpret the result in one sentence.
- (b) [7 P] A research group screens  $m = 6$  candidate biomarkers; the ordered  $p$ -values are

| $j$       | 1     | 2     | 3     | 4     | 5     | 6     |
|-----------|-------|-------|-------|-------|-------|-------|
| $p_{(j)}$ | 0.002 | 0.007 | 0.009 | 0.020 | 0.040 | 0.320 |

Control the FWER at  $\alpha = 0.05$  (i) with *Bonferroni* and (ii) with the *Holm step-down* procedure. Give the relevant thresholds, state for each method exactly which hypotheses are rejected, and explain why Holm can never reject fewer hypotheses than Bonferroni.

- (c) [6 P] Apply the *Benjamini-Hochberg* (BH) procedure with FDR level  $q = 0.10$  to the same six ordered  $p$ -values from part (b) ( $p_{(1)}, \dots, p_{(6)} = 0.002, 0.007, 0.009, 0.020, 0.040, 0.320$ ): give the thresholds  $jq/m$ , determine which hypotheses are rejected, and state precisely what “FDR control at  $q = 0.10$ ” guarantees for the resulting set of rejections.
- (d) [3 P] A pharmaceutical company runs a confirmatory phase-III trial with  $m = 4$  pre-specified primary endpoints; a positive result on any endpoint would support regulatory approval of the drug. Should they control the FWER or the FDR? Justify your recommendation.

## Solution

(a) **FWER for  $m = 25$  independent tests.**  $\text{FWER} = \Pr(\text{at least one Type I error}) = \Pr(V \geq 1)$ , where  $V$  is the number of true nulls that are (falsely) rejected. With independence and all nulls true:

$$\text{FWER} = 1 - \Pr(\text{no false rejection}) = 1 - (1 - 0.05)^{25} = 1 - (0.95)^{25} = 1 - 0.2774 = \mathbf{0.7226}.$$

Interpretation: even though each single test is performed at the 5% level, there is about a 72% chance of at least one false discovery among the 25 tests — testing many hypotheses without adjustment almost guarantees spurious findings.



**(b) Bonferroni and Holm at  $\alpha = 0.05$ .** *Bonferroni:* reject  $H_{(j)}$  iff  $p_{(j)} \leq \alpha/m = 0.05/6 = 0.00833$ .  $p_{(1)} = 0.002 \leq 0.00833$  ✓,  $p_{(2)} = 0.007 \leq 0.00833$  ✓,  $p_{(3)} = 0.009 > 0.00833$  ×.  $\Rightarrow$  Bonferroni rejects  $H_{(1)}$  and  $H_{(2)}$ : **2 rejections**.

*Holm:* compare  $p_{(j)}$  with  $\alpha/(m - j + 1)$  step by step and stop at the first failure:

| $j$                  | 1       | 2      | 3      | 4      | 5      | 6      |
|----------------------|---------|--------|--------|--------|--------|--------|
| $p_{(j)}$            | 0.002   | 0.007  | 0.009  | 0.020  | 0.040  | 0.320  |
| $\alpha/(m - j + 1)$ | 0.00833 | 0.0100 | 0.0125 | 0.0167 | 0.0250 | 0.0500 |
| reject?              | ✓       | ✓      | ✓      | stop   | —      | —      |

$0.002 \leq 0.00833$ ,  $0.007 \leq 0.0100$ ,  $0.009 \leq 0.0125$ , but  $0.020 > 0.0167$ : stop.  $\Rightarrow$  Holm rejects  $H_{(1)}, H_{(2)}, H_{(3)}$ : **3 rejections**.

*Why Holm  $\supseteq$  Bonferroni:* Holm's thresholds  $\alpha/(m - j + 1)$  are  $\geq \alpha/m$  for every  $j$  (with equality only at  $j = 1$ ), so every  $p$ -value that clears the Bonferroni bar  $\alpha/m$  also clears Holm's less stringent hurdles up to the point where Holm stops — Holm is uniformly at least as powerful, yet still controls the FWER at  $\alpha$  without any independence assumption.

**(c) Benjamini–Hochberg at  $q = 0.10$ .** Thresholds  $jq/m = j \cdot 0.10/6$ :

| $j$                   | 1      | 2      | 3      | 4      | 5          | 6      |
|-----------------------|--------|--------|--------|--------|------------|--------|
| $p_{(j)}$             | 0.002  | 0.007  | 0.009  | 0.020  | 0.040      | 0.320  |
| $jq/m$                | 0.0167 | 0.0333 | 0.0500 | 0.0667 | 0.0833     | 0.1000 |
| $p_{(j)} \leq jq/m$ ? | yes    | yes    | yes    | yes    | <b>yes</b> | no     |

$L =$  largest  $j$  with  $p_{(j)} \leq jq/m$ :  $j = 5$  (since  $0.040 \leq 0.0833$ , while  $0.320 > 0.1000$ ). BH rejects *all*  $H_{(j)}$  with  $j \leq L$ :  $H_{(1)}, \dots, H_{(5)}$ : **5 rejections**. Guarantee:  $\text{FDR} = E[V/\max(R, 1)] \leq q = 0.10$  — *on average* at most 10% of the rejected hypotheses are false discoveries. It does *not* guarantee that at most 10% of these particular 5 rejections are false, nor does it bound the probability of making any false rejection at all (that would be FWER control). Note how the three procedures grow more liberal here: Bonferroni (2)  $\subseteq$  Holm (3)  $\subseteq$  BH (5).

**(d) Confirmatory phase-III trial.** Control the **FWER** (e.g. Bonferroni or Holm). This is a *confirmatory* study whose result feeds a regulatory decision, so even a *single* false positive is costly — it could lead to approving an ineffective drug. With only  $m = 4$  pre-specified endpoints the power sacrificed by strict FWER control is small, and the strong guarantee (probability of *any* false rejection  $\leq \alpha$ ) is exactly what regulators require. FDR control would be appropriate for a large exploratory screen, not for a small confirmatory endpoint family.

## Problem 7: Cumulative essentials (Chapters 2–6)

(20 points)

Lecture slides: Chapters 2–6 (Lectures 1–8).

- (a) [4 P] True or false — justify briefly: “By choosing a sufficiently flexible model and collecting enough training data, the expected test MSE at a point can be driven all the way down to zero.”
- (b) [4 P] A logistic regression predicts whether a candidate passes a certification exam (`passed`= 1) from the number of `hours` of study,  $\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 \text{hours}$ . The model was fitted in Python (`statsmodels Logit`), with the following output:



|       | coef    | std err | $z$    | $P >  z $ |
|-------|---------|---------|--------|-----------|
| const | -3.2000 | 0.700   | -4.571 | 0.000     |
| hours | 0.8000  | 0.160   | 5.000  | 0.000     |

Using the estimated coefficient on **hours**, interpret the effect of *one additional hour of study* on the *odds* of passing (use  $e^{0.8} = 2.226$ ). Why can the effect on the *probability* of passing not be stated as a single number?

- (c) **[4 P]** Give one reason to prefer  $k$ -fold cross-validation with  $k = 5$  or  $10$  over *LOOCV*, and one reason to prefer it over a *single validation split*.
- (d) **[2 P]** You have  $p = 1,200$  SNP predictors but expect only about a dozen to be truly associated with the trait. Ridge or lasso — which penalty is more appropriate, and why?
- (e) **[6 P]** Match each scenario to exactly one method from  $\{\text{linear regression, lasso, logistic regression, random forest}\}$  (each method used exactly once) and justify each choice in one sentence:
- (i) Estimate how much one additional year of schooling raises annual earnings, with a confidence interval, from  $n = 4,000$  survey respondents and a dozen predictors.
  - (ii) Predict a continuous customer lifetime value from  $p = 3,000$  behavioural features for  $n = 250$  customers, and deliver a short interpretable list of the main drivers.
  - (iii) Predict whether a hospital patient will be readmitted within 30 days (yes/no) and report to administrators how each risk factor changes the odds of readmission.
  - (iv) Predict daily bike-share ridership from hundreds of weather, calendar and station variables with strong non-linear effects and interactions; only forecast accuracy matters.

### Solution

**(a) False.** The expected test MSE at a point decomposes as  $\text{Var}(\hat{f}) + [\text{Bias}(\hat{f})]^2 + \text{Var}(\varepsilon)$ , and the last term — the *irreducible error*  $\text{Var}(\varepsilon)$  — comes from noise in  $Y$  that no model and no amount of data can explain. It is a hard lower bound: the expected test MSE can never fall below  $\text{Var}(\varepsilon) > 0$ . Flexibility and more data can shrink bias and variance (the *reducible error*) towards 0, but not the irreducible part.

**(b) Odds ratio.** The fitted coefficient on **hours** is  $\hat{\beta}_1 = 0.8000$  (from the **Logit** output). One additional hour of study increases the log-odds of passing by 0.8, i.e. it *multiplies the odds* of passing by  $e^{0.8} = 2.226$  — the odds go up by about 123% (more than double), regardless of the current number of hours. The effect on the *probability* depends on where the candidate sits on the S-shaped logistic curve: the same change in log-odds moves  $p$  a lot near  $p \approx 0.5$  but very little when  $p$  is already close to 0 or 1 — hence no single number describes it.

**(c)  $k$ -fold CV with  $k = 5$  or  $10$ .** *vs. LOOCV:* the  $n$  LOOCV fits are trained on almost identical data sets, so their errors are highly positively correlated; averaging strongly correlated quantities reduces variance little, so the LOOCV estimate has *higher variance* than 5- or 10-fold CV (it is also computationally expensive:  $n$  fits instead of  $k$ , absent shortcut formulas). *vs. a single validation split:* the validation estimate depends strongly on the random split and overestimates the test error because only part of the data is used for training (*bias*);  $k$ -fold CV trains on  $(k - 1)/k$  of the data and averages over  $k$  folds, giving a less biased and more stable estimate.

**(d) Lasso.** The  $\ell_1$  penalty sets many coefficients *exactly to zero* and thus performs variable selection — ideal when the truth is sparse (about a dozen signals among 1,200 SNPs), and it yields an interpretable small model. Ridge only shrinks coefficients but keeps all  $p = 1,200$  predictors in the model, so it tends to lose when the vast majority of true coefficients are zero.

**(e) Matching.**

- (i) **Linear regression:** quantitative response, low dimension, goal is *inference* (effect size with confidence interval) — exactly what OLS with standard errors provides.
- (ii) **Lasso:**  $p \gg n$  (3,000 features, 250 customers) rules out OLS; the  $\ell_1$  penalty regularises and returns a short list of the influential drivers.
- (iii) **Logistic regression:** binary response with interpretable coefficients ( $e^{\hat{\beta}_j}$  = odds ratios) — suitable for reporting risk factors to administrators.
- (iv) **Random forest:** captures non-linearities and interactions automatically and is accurate with many heterogeneous predictors; interpretability is not required and only accuracy matters.

---

*End of the examination — good luck!*